APERTURA

Desarrollo de software asistido por IA

INSTITUTO POLITÉCNICO FORMOSA
"Dr. Alberto Patrocelo Zorrilla"

Modelos · agentes · herramientas · contexto · memoria · MCP · skills · OpenCode

De pedir código a orquestar trabajo de ingeniería

Comprender qué hace la IA, con qué permisos y cómo verificar el resultado

INSTITUTO POLITÉCNICO FORMOSA
"Dr. Alberto Marcelo Zorrilla"

Resultados de aprendizaje

IDENTIFICAR

Distinguir modelos, asistentes, agentes, herramientas y protocolos.

EXPLICAR

Describir el ciclo de un agente y la función del contexto y la memoria.

INTEGRAR

Reconocer cómo MCP, skills y reglas amplían un entorno de trabajo.

EVALUAR

Revisar código generado y justificar su aceptación o rechazo.

Un ecosistema, varias capas

Modelo

Genera y razona sobre tokens

→

Host / agente

Gestiona sesión y acciones

→

Herramientas

Archivos, terminal, tests

→

Integraciones

MCP, servicios externos

Un chatbot, un IDE asistido y un agente de terminal pueden usar modelos similares, pero exponen capacidades diferentes.

Modelo ≠ producto ≠ agente

MODELO

- Procesa entradas y genera salidas.
- No accede solo a tu sistema de archivos.
- El límite de contexto y el proveedor importan.

AGENTE / PRODUCTO

- Orquesta modelo, herramientas y estado.
- Puede inspeccionar, modificar o ejecutar.
- Depende de permisos, entorno y configuración.

Opciones de modelos: tres decisiones

PROVEEDOR

API remota: disponibilidad, precio, límites y tratamiento de datos.

MODELO LOCAL

Ejecución en hardware propio: control, memoria y rendimiento.

CAPACIDAD

Adecuación para código, razonamiento, herramientas y contexto.

COSTO REAL

“Gratuito” puede incluir cuotas, cambios de catálogo o restricciones.

En OpenCode, comprobar la oferta vigente con /models antes de la demostración.

INSTITUTO POLITÉCNICO FORMOSA
"Dr. Alberto Marcelo Zorrilla"

Tokens, contexto e inferencia

TOKENS

Unidades de texto procesadas. Las entradas y las salidas consumen capacidad.

VENTANA DE CONTEXTO

Información que el modelo puede considerar en una llamada. No equivale a memoria permanente.

INFERENCIA

Ejecución del modelo para producir una respuesta o llamadas a herramientas.

LÍMITES

Más contexto no garantiza mejor resultado; el ruido y la información obsoleta afectan.

INSTITUTO POLITÉCNICO FORMOSA
"Dr. Alberto Marcelo Zorrilla"

¿Qué es un agente de desarrollo?

Sistema que utiliza un modelo y capacidades operativas para avanzar sobre objetivos, observar resultados y decidir siguientes pasos.

Objetivo
Lo define el humano

→

Decisión
El modelo propone

→

Acción
Se ejecuta una herramienta

→

Observación
Se lee el resultado

No todos los agentes operan sin supervisión; el grado de autonomía es configurable.

INSTITUTO POLITÉCNICO FORMOSA
"Dr. Alberto Marcelo Zorrilla"

Anatomía de un agente

MODELO

Interpretación, generación y elección de acciones.

INSTRUCCIONES

Rol, prioridades, restricciones y políticas.

CONTEXTO / ESTADO

Historial, archivos, resultados y decisiones relevantes.

HERRAMIENTAS / PERMISOS

Qué puede consultar, editar, ejecutar y con qué confirmaciones.

CONTROL DE EJECUCIÓN

Bucle de trabajo, límites, errores y cancelación.

SUPERVISIÓN

Revisión humana, trazabilidad, pruebas y aprobación.

El bucle del agente

1. Analiza

Lee la tarea y el proyecto

→

2. Planea

Descompone el problema

→

3. Actúa

Invoca herramienta

→

4. Comprueba

Lee salida y corrige

El resultado de una herramienta vuelve al contexto de la siguiente decisión.

Asistente, agente y subagente

ASISTENTE

Ayuda a responder, explicar y sugerir cambios.

AGENTE

Ejecuta un proceso con pasos, observaciones y herramientas.

SUBAGENTE

Agente especializado que recibe una tarea acotada y devuelve resultados.

MULTIAGENTE

Varios roles o agentes coordinados; más complejidad no implica mejor calidad.

Permisos y niveles de autonomía

MODO RESTRINGIDO

- Lee e inspecciona archivos.
- Propone diffs sin aplicarlos.
- Solicita aprobación para acciones sensibles.

MODO OPERATIVO

- Puede editar y ejecutar comandos.
- Debe trabajar en un entorno controlado.
- Requiere límites, revisión y trazabilidad.

Evitar acceso a credenciales reales y comandos destructivos en una demostración educativa.

¿Qué información ve un modelo?

INSTRUCCIONES

Reglas de sistema, del agente y del repositorio.

CONVERSACIÓN

Mensajes, plan, decisiones y resultados previos.

CÓDIGO

Archivos que el agente decide leer o recibe como entrada.

HERRAMIENTAS

Salidas de terminal, tests, búsquedas y servicios externos.

Contexto no es memoria

CONTEXTO

- Información activa en una interacción.
- Puede truncarse, resumirse o renovarse.
- Relevancia > cantidad de texto.

MEMORIA

- Información conservada o recuperable.
- Puede existir en archivos, resúmenes o DB.
- Debe diseñarse: no es automática en todo agente.

Tres estrategias para recuperar información

ARCHIVOS PERSISTENTES

Convenciones, ADRs y decisiones en documentos del repositorio.

RESÚMENES

Compactación del historial para recuperar objetivos y acuerdos.

RAG

Búsqueda de fragmentos relevantes antes de responder.

EMBEDDINGS

Representaciones vectoriales útiles para recuperación semántica, no “recuerdos” por sí mismos.

Context engineering en un repositorio

Seleccionar
Identificar tarea y archivos

→

Reducir ruido
Excluir material ajeno

→

Estructurar
Proveer reglas y criterios

→

Verificar
Corregir contexto erróneo

Un buen prompt con contexto equivocado sigue siendo una instrucción deficiente.

¿Para qué existe MCP?

Model Context Protocol define interfaces para conectar aplicaciones de IA con fuentes de datos y capacidades externas.



MCP: herramientas, recursos y prompts

TOOLS

Operaciones invocables: buscar, consultar, crear o modificar (según permisos).

RESOURCES

Datos y contenidos que el servidor puede ofrecer al cliente.

PROMPTS

Plantillas de interacción que un servidor puede exponer.

LÍMITE CLAVE

El protocolo no convierte un servicio inseguro en seguro: importan permisos y confianza.

Ejemplo: documentación externa

Consulta
"Revisá esta API"

→

Agente
Necesita docs actuales

→

MCP
Llama herramienta de búsqueda

→

Resultado
Lee y cita lo encontrado

MCP no es el modelo; tampoco reemplaza la validación de los datos recibidos.

Ejemplo de configuración de MCP

```
// Ejemplo conceptual de configuración de OpenCode
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "documentacion": {
      "type": "local",
      "command": ["npx", "-y", "paquete-servidor-mcp"],
      "enabled": true
    }
  }
}
// Sustituir el paquete por un servidor confiable y verificado.
```

Demostración: servidor y dependencias deben instalarse y verificarse antes de la clase.

Una skill no es una herramienta

SKILL

- Instrucciones de cómo hacer una tarea.
- Puede incluir scripts y referencias.
- Puede cargarse cuando se la necesita.

TOOL / MCP

- Capacidad para ejecutar una operación.
- Puede acceder a datos o sistemas reales.
- Su uso implica permisos y controles.

Estructura de SKILL.md

```
---
name: revisar-api-express
description: Revisar estructura y riesgos de una API Express
---

# Procedimiento
1. Identificar rutas, controllers, services y repositories.
2. Revisar validación y manejo de errores.
3. Detectar dependencias rígidas y duplicación.
4. Ejecutar tests disponibles antes de recomendar cambios.
5. Informar hallazgos con archivo, causa y propuesta.
```

Ejemplo didáctico; la ubicación y activación de skills depende de la herramienta.

AGENTS.md: reglas del proyecto

```
# Reglas del repositorio
- Stack: Express + TypeScript + Vitest.
- Mantener separadas rutas, controladores y servicios.
- No editar archivos .env ni exponer secretos.
- Ante cambios de negocio, agregar tests unitarios.
- Antes de finalizar, ejecutar npm test y npm run build.
- Resumir archivos modificados y decisiones tomadas.
```

AGENTS.md define instrucciones; no equivale a memoria automática de todas las conversaciones.

Cuándo usar cada componente

PROMPT

La solicitud puntual: “implementá un endpoint...”

AGENTS.md

Las reglas que deben orientar tareas del repositorio.

SKILL

Un procedimiento reutilizable: “auditar API Express”.

MCP

La conexión a capacidades externas: documentación o servicios.

Del pedido informal a la especificación

PEDIDO VAGO

- “Creá un CRUD de empleados”.
- Ambigüedad de reglas y errores.
- Difícil comprobar que está terminado.

ESPECIFICACIÓN

- Endpoints y reglas verificables.
- Estructura, errores y restricciones.
- Criterios de aceptación + pruebas.

Una especificación breve y útil

```
Funcionalidad: Registrar empleado
POST /empleados
Datos: nombre, email, salarioBase
Reglas:
 - email válido y único
 - salarioBase > 0
 - 201 al crear; 400/409 según el error
Diseño:
 - controller / service / repository
Validación:
 - pruebas para éxito y casos inválidos
 - no modificar otros endpoints
```

Trabajo recomendado con un agente

Especificar
Objetivo y aceptación

→

Planificar
Archivos y dependencias

→

Implementar
Cambios acotados

→

Verificar
Tests + diff + revisión

La IA puede proponer un plan; el desarrollador decide si lo aprueba.

INSTITUTO POLITÉCNICO FORMOSA
"Dr. Alberto Marcelo Zorrilla"

El test también puede estar equivocado

PRUEBA UNITARIA

Valida lógica aislada y casos límite.

PRUEBA DE INTEGRACIÓN

Verifica interacción real entre capas o dependencias.

REVISIÓN DE DIFF

Detecta cambios no solicitados, regresiones y secretos.

CRITERIO HUMANO

Contrasta código y pruebas con los requisitos originales.

Cuatro errores habituales

ALUCINACIÓN

API o biblioteca inventada, firma incorrecta, versiones incompatibles.

DEUDA TÉCNICA

Abstracciones innecesarias, duplicación y sobreingeniería.

VULNERABILIDAD

Validación débil, SQL injection, secretos o permisos inseguros.

FALSA CONFIANZA

Tests que solo comprueban lo que el propio código ya supone.

Prompt injection y herramientas

Entrada
README, issue, web o docs

→

Contenido hostil
Instrucciones camufladas

→

Agente
Confunde datos con órdenes

→

Impacto
Acción fuera del objetivo

Defensa: separar instrucciones de datos, restringir herramientas y pedir confirmación para acciones sensibles.

Checklist antes de aceptar cambios

- ¿Resuelve exactamente el requisito solicitado?
- ¿Mantiene separación de responsabilidades y diseño coherente?
- ¿Introduce paquetes, permisos, secretos o cambios no pedidos?
- ¿Compila y supera pruebas relevantes, incluidos casos límite?
- ¿Puedo explicar y defender cada decisión importante?

Laboratorio: OpenCode sobre una API

PREPARAR

Proyecto Express + TypeScript, Git limpio, tests iniciales.

INSPECCIONAR

Pedir al agente que describa capas y riesgos sin editar.

PLANIFICAR

Crear endpoint de empleados con aceptación verificable.

VALIDAR

Revisar diff y correr npm test / npm run build.

Seleccionar antes un modelo gratuito disponible en /models; probar credenciales y límites de uso.

Demostración: inspección sin cambios

Prompt 1
“Inspeccioná el repositorio. Explicá cómo fluyen las
solicitudes desde routes hasta repository. Identificá
tres riesgos de mantenibilidad. No modifiques archivos.”

Observar
✓ Archivos consultados
✓ Herramientas invocadas
✓ Evidencia citada por el agente
✓ Errores y suposiciones

Demostración: plan y aprobación

Prompt 2
“Necesitamos POST /empleados con email único.
Leé AGENTS.md y prepará un plan por archivos,
criterios de aceptación y pruebas. No implementes
hasta que apruebe el plan.”

Debate
¿Faltan reglas de negocio?
¿Qué parte del plan cambiarían?

Demostración: implementación y evidencia

```
Prompt 3
“Implementá únicamente el plan aprobado.
 Agregá pruebas de éxito, datos inválidos y duplicados.
 Ejecutá pruebas y build. Mostrá un resumen del diff,
 comandos ejecutados y limitaciones encontradas.”

Verificación humana:
 - git diff
 - npm test
 - npm run build
```

Extensión opcional: MCP y skill

MCP

Conectar un servidor de documentación probado e invocar una herramienta verificable.

SKILL

Cargar una revisión específica de Express desde SKILL.md.

COMPARAR

¿Qué vino del modelo, de archivos del proyecto y de herramientas externas?

CONTROL

Mostrar permisos y explicar qué acciones requieren aprobación.

¿Qué se evalúa en un desarrollador?

No solo escribir código, sino definir criterios, dar contexto, gobernar herramientas y verificar resultados.

La IA propone y ejecuta según sus capacidades. La responsabilidad técnica sigue siendo humana.

Preguntas para discusión

- ¿Cuándo preferirían un chat frente a un agente con herramientas?
- ¿Qué información nunca incluirían en el contexto de un agente?
- ¿En qué caso usarían MCP y en cuál una skill?
- ¿Qué evidencia pedirían antes de desplegar código generado?
- ¿Qué riesgo aumenta al otorgar permisos de terminal y escritura?

Documentación para continuar aprendiendo

OPENCODE

opencode.ai/docs · Agents · Skills · Rules · MCP Servers

MCP

modelcontextprotocol.io · Conceptos y arquitectura

AGENT SKILLS

agentskills.io · Estructura y especificación de skills

PRUEBAS

vitest.dev · Documentación y ejemplos de testing